

Day 3

8.5 Step by step

1. Record exactly what triggered the alert: the process, the connection, and the behaviour that was flagged. Quote the alert.
2. Working from the five captures, write down the host's state at the time of capture: what was running, what it was connected to, and who was logged in.
3. List every indicator of compromise visible so far - process names, IP addresses, ports, file paths, account names. Use CyberChef for the addresses.
4. Find the one login in Capture 3 that should not be there, and say what makes it stand out. Compare its source address against everything else in the captures.
5. Document your evidence-handling steps in the order they were performed, and explain why volatile data was captured before anything else. Reference the volatility table in Section 8.2.

Type	Value	Where you saw it	Why it is suspicious
Process	apache-worker	Capture 1	Apache-worker is running as root. Apache-worker manages incoming client requests. In this case, it is launched through /bin/bash, which is the command-line.
IP address	185.220.101.47	Capture 2, 3, 4	External IP-address shown attached to root login and apache-worker
Port	8443	Capture 2	Alternate HTTPS
File path	/usr/local/bin/apache-worker	Capture 1 and 5	Suspicious path using account that is being accessed externally
Account	root	Capture 3	Evidence of root login from external IP address
Scheduled task	<code>*/5 * * * * root /usr/local/bin/apache-worker >/dev/null 2>&1</code>	Capture 5	Potential hiding of commands and persistence

9. Day 3, Part 2: Evidence Integrity

9.3 Verify a hash yourself in CyberChef

You will now perform the acquisition-and-verification cycle on a small evidence block, so you have done it rather than only read about it.

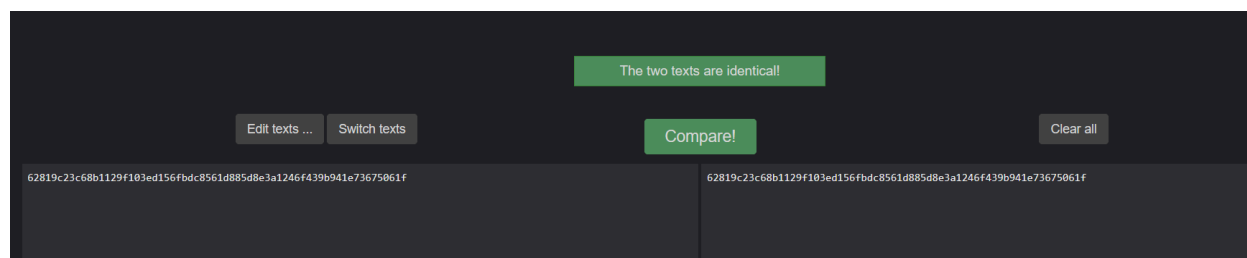
1. Open gchq.github.io/CyberChef. Clear the Recipe panel if anything is left from Section 8.
2. Search Operations for **SHA2** and drag **SHA2** into the Recipe. It is in the **Hashing** category.
3. In the SHA2 operation, make sure **Size** is set to **256**. That is the default.

4. Type this exactly into the Input panel, on one line, with no space or line break before or after it:

```
MTB-IR-2026-0902-EVIDENCE-BLOCK-01
```

5. Compare your Output against the recorded value below. They must match character for character.

```
62819c23c68b1129f103ed156fbd8561d885d8e3a1246f439b941e73675061f
```



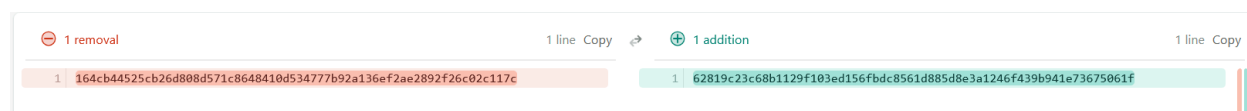
9.4 Prove that tampering is detectable

1. Change the Input to end in `02` instead of `01`, so it reads `MTB-IR-2026-0902-EVIDENCE-BLOCK-02`.
2. Look at the new Output. Confirm it matches the value below.

```
164cb44525cb26d808d571c8648410d534777b92a136ef2ae2892f26c02c117c
```

3. Compare the two hashes side by side. You changed one character out of 34. Write down how many characters of the hash stayed the same.
 - All characters are different
4. Write two sentences explaining what this demonstrates about why a hash recorded at acquisition is sufficient to prove evidence has not been altered.

A hash recorded at acquisition is important for digital forensics because it proves that the data caught at acquisition was not tampered with or altered. Even just one character changing causes the entire hash to change.



10.4 Step by step

1. Copy the incident window timeline into a new spreadsheet tab called **Timeline**. Split it on commas so each field lands in its own column.
2. Add a column called **Interpretation**. Write one plain-English sentence for every row explaining what that entry means. Not "passwd was modified" - say what modifying it at that moment achieves for an attacker.
3. Mark every row whose Type is `macb`. These are files created during the incident. Group them: which are payload, which are persistence, which are access?

4. Two files were created 27 seconds apart at 02:25:57 and 02:26:30. Explain what those two together accomplish that neither achieves alone.
5. Look at the two entries at 02:41:19 and 02:41:22. One file has content, the other is zero bytes. Explain what the attacker was doing and why one attempt worked while the other did not.
 - The attacker looked to see if the /home/mtbadmin/root_history file had any data. The reason /root/.bash_history has data is because it was not deleted, unlike /mtbadmin/root_history

6.

- The attacker accessed the /usr/local/bin/apache-worker folder. This is a red flag because the suspicious user is accessing a path that is used for installing user-executable files that are not managed by the default package manager. The next log shows that the suspicious user created a cronjob that is used to rotate, delete, and compress old logs. The attacker might be trying to hide their tracks with this custom job.

7. Compare the incident window against the baseline in Section 10.2. Write two sentences on what makes the incident activity obviously abnormal for this host.
 - The incident activity is obviously abnormal for this host because the incident log shows access to critical files such as /etc/shadow and authorized_keys. Additionally, suspicious custom jobs that rotate logs were created as seen by the incident activity log.

11. Day 3, Part 4: Log Correlation and Indicator Analysis

11.5 Decoding the suspicious file

The PHP file contains a base64-encoded string. Encoding is not encryption - it is a way of representing data as plain text, and it is trivially reversible. Attackers use it constantly to make payloads harder to spot in a casual read.

1. In CyberChef, clear the Recipe panel.
2. Search Operations for **From Base64** and drag **From Base64** into the Recipe. It is in the **Data format** category.
3. Paste the encoded string into the Input panel. It is printed across two lines in Section 11.4 for page width - join it into one line with no space where it wraps.
4. Read the Output. Record what it decodes to.
5. Write two sentences on why an attacker would encode a payload this way, and why encoding provides no actual security.

MTB-LAB-ARTIFACT: simulated webshell payload, no executable content

An attacker would encode the payload this way so that automated tools may not be able to detect the malicious payload, unless specifically modified to decode it. It is a simple way to try to avoid detection, but provides no security.

11.6 Building the consolidated IOC list

An IOC list is the deliverable another team acts on. Its whole purpose is to let someone who was not part of your investigation search their own systems for the same activity. That means it must be precise, complete, and safe to circulate.

1. Collect every indicator from the timeline, the logs, the recovered files and the acquisition record. Use CyberChef's **Extract IP addresses** on the log sections to make sure you missed nothing.
2. Sort them into types: network, file, hash, account, persistence.
3. For each indicator, record where you found it. An indicator without a source is not evidence.
4. Defang the network indicators before circulating the list. In CyberChef, search for **Defang IP** and drag **Defang IP Addresses** into the Recipe. Paste your address list in.

11.7 Record: consolidated IOC list

Type	Indicator (defanged)	Source	What it detects
Network	185[.]220[.]101[.]47	Apache access log; Capture 2; Capture 3; Capture 4	Attacker IP communicating with and accessing MTB-WEB01
Network	185[.]220[.]101[.]47:8443	Alert; Capture 2; Capture 4	Outbound beacon connection to attacker IP on port 8443
File path	/var/www/html/uploads/style.php	File system timeline; Apache access log; recovered files	Suspicious uploaded PHP webshell
File path	/usr/local/bin/apache-worker	File system timeline; Capture 1; Capture 5; system log	Suspicious executable used for outbound beaconing
File hash	2dfc22e2fb93187b7537d9fffec52ecc94fd6416c84b78e68ff7a025e975c20c	Acquisition record	Identifies the exact style.php file
File hash	1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b	Acquisition record	Identifies the exact apache-worker file
Account	svc-backup	Authentication log; file system timeline; root .bash_history	Unauthorized UID 0/root-equivalent backdoor account
Persistence	/etc/cron.d/apache-logrotate	File system timeline; system log; root .bash_history	Cron persistence that repeatedly launches apache-worker
Other	Attacker SSH public key (attacker@blackfin)	/root/.ssh/authorized_keys recovered file	Unauthorized SSH key providing persistent root access

AI was used in this section to fill out the table. All findings were originally completed by me and only me.

12. Day 3, Part 5: Root Cause and Forensic Report

12.1 Finding the root cause

Root cause is not the webshell, and it is not the beacon. Those are consequences. Root cause is the specific weakness that let an unauthenticated attacker on the internet get their first foothold on a bank's DMZ web server.

1. Look at the first request in the access log that returned a 200 with a body, from the external address, using an unusual URL structure. Note the time.

```
185.220.101.47 - - [02/Sep/2026:02:12:33] "GET /cgi-bin/.%2e/.%2e/.%2e/.%2e/etc/passwd" 200 622 "curl/7.81.0"
```

2. Match that request pattern against the findings from your Day 1 assessment. Which finding describes exactly this technique against exactly this software version?

The finding that matches this technique exactly against the software version is cve-2021-42013

3. Record the finding ID and the CVE. Check your Day 2 work: where did that finding rank after you applied the KEV cross-reference, and where would it have ranked on CVSS alone?
 - It is ranked at the top. If it did not have a KEV value, it would be lower. If ranked only by CVSS, it would still be at the top.
4. Identify the second vulnerability in the chain - the one that took the attacker from the web server account to root. It is also on your Day 1 list.
 - The second vulnerability in the chain, CVE-2021-4034, is what took the attacker from the web server to the root.
5. Note the configuration on MTB-WEB01 that made the first vulnerability exploitable. It is printed in the evidence pack in Section 5.3. The software version alone was not sufficient.
 - Apache webserver was running version 2.49.49, which is in-range of the vulnerable versions for CVE-2021-42013. Specifically, \$ grep -A2 'ScriptAlias' /etc/apache2/sites-enabled/000-default.conf
 - ScriptAlias /cgi-bin/ /usr/lib/cgi-bin/
 - <Directory />
 - Require all granted
 - Granted access for everyone to cgi-bin

12.3 Prevention and monitoring recommendations

Recommendations must be specific enough that someone could implement them from your report alone. Cover all three of these:

- **Patching** - name the packages and the versions to upgrade to, and address why a host built in May was still running software with a publicly known, actively exploited flaw in September.
 - Apache webserver should be updated from 2.49.49 to 2.4.51 or later.
 - Pkexec should be updated to a version containing the Pwnit fix

- **Configuration** - name the specific Apache directives that made the flaw exploitable and what they should be set to instead.
 - The specific Apache directive that caused this incident to happen was allowing access to cgi-bin for everyone. Script-alias should be removed or disabled.
- **Detection** - at least one recommendation must be an endpoint protection or monitoring improvement that would have caught this faster. The compromise happened at 02:15 and was detected at 09:47. Explain what would have shortened that gap and by roughly how much.
 - File integrity monitoring would've caught the edits that the attacker made to root/.ssh_authorized_keys. This would've shortened the length to roughly only a few minutes.

Detection is where most of the value is

The attacker was inside for seven and a half hours before anyone noticed, and they were only noticed because the beacon was noisy and predictable. A quieter one would still be running.

For each of the three persistence mechanisms you identified, ask what would have alerted on it at the moment it was created rather than hours later. Write those down as concrete detections, not as "improve monitoring".

Section	Contents
Executive summary	Three to five sentences for a non-technical stakeholder. What happened, what was at risk, what has been done.
Evidence handling	How evidence was preserved, in what order, and how integrity was verified. Include your hashes.
Incident timeline	The merged timeline from Section 12.2, with confidence marked.
Root cause	The vulnerability chain, the CVEs, the configuration that enabled it, and the link to your Day 1 finding IDs.
Indicators of compromise	Your defanged IOC list from Section 11.7.
Recommendations	Patching, configuration and detection, each specific and actionable.

The specific incident occurred with our website-hosting server. An attacker was able to exploit a vulnerability in the system that allowed them admin access. This has been remediated after a comprehensive effort by the analyst team to fix the issue.

The hashes that were found at acquisition have been checked extensively, and they do match. This means that evidence was unchanged and untampered.

Timeline of events:

Time (UTC)	Event	Evidence source	Confidence
2:08:14	Nmap scan	Apache Access Log 11.1	Inferred
2:12:33	GET command for /etc/passwd	Apache Access Log 11.1	Confirmed
2:15:22	Styles.php was modified, accessed, changed, and born	Incident Window 10.3	Confirmed
2:17:40	Style.php sent a POST request	Apache Access Log 11.1	Confirmed
2:20:47	/etc/passwd was accessed	Apache Access Log 11.1	Confirmed
2:23:11	/etc/passwd was accessed	Incident Window 10.3	Inferred
2:24:48	/root/.ssh/authorized_keys was accessed	Incident Window 10.3	Inferred
2:25:57	/usr/local/bin/apache-worker was created	Incident Window 10.3	Inferred
2:26:30	/etc/cron.d/apache-logrotate was created	Incident Window 10.3	Inferred
2:34:55	/tmp/.ICE-unix was modified and accessed	Incident Window 10.3	Inferred
2:41:19	/root/.bash_history was accessed	Incident Window 10.3	Inferred
9:47:12	Periodic outbound connection to 185.220.101.4	The Alert 8.1	Confirmed

Root cause

The root cause of this incident was the Apache 2.4.49 vulnerability. This vulnerability allows an attacker to traverse the paths of an Apache webserver running an affected version. Additionally, a misconfiguration allowing cgi-bin to be accessed by any account was important root cause in this instance (`ScriptAlias /cgi-bin/ /usr/lib/cgi-bin/` was enabled while `<Directory />` used `Require all granted`.) Finally, Pwnkit (CVE-2021-4034) was utilized to escalate privileges.

Indicators of Compromise

Type	Indicator (defanged)	Source	What it detects
Network	185[.]220[.]101[.]	Apache access log; Capture 2; Capture 3;	Attacker IP communicating with and accessing

	.]47	Capture 4	MTB-WEB01
Network	185[.]220[.]101[.]47:8443	Alert; Capture 2; Capture 4	Outbound beacon connection to attacker IP on port 8443
File path	/var/www/html/uploads/style.php	File system timeline; Apache access log; recovered files	Suspicious uploaded PHP webshell
File path	/usr/local/bin/apache-worker	File system timeline; Capture 1; Capture 5; system log	Suspicious executable used for outbound beaconing
File hash	2dfc22e2fb93187b7537d9fffec52ecc94fd6416c84b78e68ff7a025e975c20c	Acquisition record	Identifies the exact style.php file
File hash	1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b	Acquisition record	Identifies the exact apache-worker file
Account	svc-backup	Authentication log; file system timeline; root .bash_history	Unauthorized UID 0/root-equivalent backdoor account
Persistence	/etc/cron.d/apache-logrotate	File system timeline; system log; root .bash_history	Cron persistence that repeatedly launches apache-worker
Other	Attacker SSH public key (attacker@blackfin)	/root/.ssh/authorized_keys recovered file	Unauthorized SSH key providing persistent root access

Recommendations

- Patching** - name the packages and the versions to upgrade to, and address why a host built in May was still running software with a publicly known, actively exploited flaw in September.
 - Apache webserver should be updated from 2.49.49 to 2.4.51 or later.
 - Pkexec should be updated to a version containing the Pwnit fix
- Configuration** - name the specific Apache directives that made the flaw exploitable and what they should be set to instead.
 - The specific Apache directive that caused this incident to happen was allowing access to cgi-bin for everyone. Script-alias should be removed or disabled.
- Detection** - at least one recommendation must be an endpoint protection or monitoring improvement that would have caught this faster. The compromise happened at 02:15 and was detected at 09:47. Explain what would have shortened that gap and by roughly how much.

- File integrity monitoring would've caught the edits that the attacker made to `root/.ssh_authorized_keys`. This would've shortened the length to roughly only a few minutes.

14. Reflection Questions

Answer briefly in your own words, three to five sentences each. These are graded on reasoning, not length.

14.1 Days 1 and 2

1. Why can a vulnerability assessment not stop at the raw scanner output? Give a specific example from this lab of what manual validation changed.
 - A vulnerability scanner
2. Give an example from this scan where CVSS score alone would have led you to the wrong priority order, and explain what threat intelligence corrected.
 - CVE-2019-0708, was found on the scan. It would've led me to the wrong priority order because it was a false-positive. The version of Windows Server that it affects is older than the one currently running on the host. Threat-intelligence corrected this by verifying each critical CVE.
3. All three false positives in this scan were in the KEV catalog. Explain why that combination is more dangerous to a security team than a false positive that is not in KEV.
 - This is more dangerous to a security team because KEV stands for Known Exploited Vulnerabilities, which renders it an urgent matter to be fixed. If it was not in the KEV, the priority for it to be remediated drops.
4. Why is a compensating control still valuable even when it does not fix the underlying vulnerability?
 - Because sometimes you might not be able to fix the underlying vulnerability. Legacy applications use old ports and protocols that are important for business. Compensating controls show that the applications are still important and that they can be secured with controls that can reduce the chance of an attack.

14.2 Day 3

1. Why does the order you collect evidence in matter, and what specifically would have been lost if the disk had been imaged before the running processes and network connections were captured?
 - Order of collection matters because of state. Artefacts in volatile memory would be lost if the machine was powered off. Leaving it on preserves the evidence and allows the forensics team to discover more about what happened.
2. How did correlating the host logs with the monitoring alert change or confirm your understanding of what happened?
 - Correlating the host logs with the monitoring logs allowed me to infer or confirm my findings. Some parts of the log were backed-up by what was seen in another log, thus ensuring that what I was seeing was most likely accurate and important.
3. Why is identifying the root cause more valuable to the bank than simply removing the malicious process and the backdoor account?

- The root cause is extremely valuable because it shows that the team actually discovered what happened and fixed it to prevent another incident from occurring. If they had just removed the process and the account, another person could easily exploit the vulnerability since it wasn't patched.
- 4. The attacker cleared one shell history file and missed another. What does that tell you about relying on any single evidence source, and how did the timeline let you detect the deletion even though the contents were gone?
- If you only relied on a single evidence source, you might miss important details. If I had only looked at one log, I wouldn't have discovered that the attacker missed deleting a log file. If I did not know, I might not have been able to identify what exactly happened.